第4讲 数据布局与内存安全

ICS 第一次小测复习 · 发布版 · 2026-10-13 · 独立答案另册

1 先把问题变成地址计算
2 结构体要满足成员与整体的对齐
3 联合是共享同一片字节
4 浮点数在汇编里走另一组寄存器
5 程序内存布局与越界
6 ROP 只需先看懂 ret 链
7 独立练习
8 收尾与来源

这是复习计划的第4天，对应课程第7、8讲，并复用前面学过的寻址和调用栈。读完后，你应能把一个 C 对象画成字节，算出成员或元素的位置，并解释越界写入为什么会影响程序控制流。

本讲采用课程的 x86-64、Linux、System V ABI：char 1 字节，short 2，int/float 4，long/double/普通指针 8；默认小端和常见自然对齐，不使用 packed。下文的大小和偏移依赖这些前提，不是所有 C 平台的固定规则。

1 先把问题变成地址计算

前一讲中，`movq (%rdi), %rax` 的意思是：把 rdi 里的数当作地址，从那里读取八字节。现在遇到 `a[i]`、`p->value`，机器仍然做这件事；区别只是先算出应该访问哪个地址。

读本讲前，用自己的话回答：`%rdi`、`(%rdi)`、`8(%rdi)` 分别是什么？前者是寄存器内容，后两者分别访问地址 rdi 和 rdi+8 处的内存。若把这三者混淆，先回第1讲的寻址部分。

一维数组就是等宽格子

```
int a[4] = {11, 22, 33, 44};
```

如果 a 的首地址是 0x1000，每个 int 占四字节：

图4-1 数组下标与字节地址。 从左到右地址增加；一个方框代表一个 int，而不是一个字节。



所以 `&a[i] = 首地址 + i × sizeof(int)`。a+1 跨过一个 int，增加四字节，不是增加一个字节；先转换成 `char *` 再加一，才跨过一个字节。

表达式	含义	是否需要读取数组元素
a+i、&a[i]	第 i 个元素的地址	否
a[i]、*(a+i)	第 i 个元素的值	是
sizeof(a)	整个数组的字节数，此处 16	否
sizeof(a[0])	一个元素的字节数，此处 4	否

在大多数表达式里，数组名会转换为指向首元素的指针；**数组本身不是指针变量**。sizeof(a) 和 &a 是理解这一点最有用的例外。函数形参写 int a[4] 时会调整为指针参数，因此不能在函数内用 sizeof(a) 求传入数组的总大小。

二维数组先跨行 再跨列

```
int b[2][3] = {{10, 11, 12}, {20, 21, 22}};
```

图4-2 二维形状与实际连续存储。 图的上半部分方便理解下标，下半部分才是内存的排列顺序。



如果每行 C 个元素，每个元素 K 字节：

&b[i][j] = B + K × (i × C + j)。

先乘列数 C，算出前 i 行共有多少元素；加 j 找到本行第 j 个；最后乘 K 变成字节偏移。**行号乘的是列数，不是总行数。**

例 D4-E1 从 C 下标到机器地址

助手补充例。long a[3][5] 首地址是 0x2000，求 a[1][3] 的地址，并解释 a+1。

1. 每个 long 八字节，每行五个，所以一行占 40 字节。

2. `[1][3]` 前面有 `1×5+3=8` 个元素，偏移 `8×8=64=0x40` 字节。
3. 地址是 `0x2040`。这个地址处的八字节是什么数，题面没有给出，不能凭地址猜值。
4. `a` 转换为“指向一行的指针”，`a+1` 增加 40 字节，结果是 0x2028。

若 i 在 rdi、j 在 rsi，计算元素下标 `5*i+j` 可写成：

```
leaq (%rdi,%rdi,4), %rax  # rax = 5*i，只做算术
addq %rsi, %rax           # rax = 5*i+j
# 再以8为比例访问 long 元素；首地址须另有寄存器或符号提供
```

当列数 n 在运行时才知道时，公式仍是 `i*n+j`，只是不能把 n 烘焙成固定常数。例如 `int get(size_t n, int a[][n], size_t i, size_t j)` 中，需要在运行时乘 n。这不是“教材中动态分配栈帧”的同一个问题。

别把指针数组当成二维数组

声明	从变量名向外读	+1 的步长
`int a[2][3]`	a 是两行，每行三个 int	表达式 `a+1` 跨一行，12 字节
`int *p[3]`	p 是三个指针组成的数组	`p+1` 跨一个指针，8 字节
`int (*q)[3]`	q 是指向三个 int 的数组的指针	`q+1` 跨一行，12 字节

`a[i][j]` 可以直接用行宽计算地址；`p[i][j]` 通常先读取指针 p[i]，再从它指向的地方访问第 j 个 int。p 的三行可以位于不相邻的区域，因此 `int **` 不能直接替代 `int (*)[3]`。

数组题先写类型，再算步长。看到 `sizeof` 时，先判断对象是数组还是指针，不能仅凭表达式长得像地址就都填八。

2 结构体要满足成员与整体的对齐

这一节要解决两个问题：结构体里的每个成员放在哪里，整个结构体占多少字节。成员大小之和不一定等于结构体大小，因为为了让成员从合适的地址开始，可能需要在中间和末尾留空。“对齐”约束的是存放地址，不是成员的数值。

> Q027：结构体对齐在讲什么？为什么成员合计15字节，结构体却占24字节？

结构体的字段按声明顺序放置，但中间可能留空。空位叫填充，不是隐藏变量，也不保证值为零。

常见自然对齐规则下：char 对齐 1，short 对齐 2，int/float 对齐 4，long/double/指针对齐 8。对齐为 K 的字段，其起始地址应是 K 的倍数；结构体整体大小是最大成员对齐要求的倍数，保证结构数组的下一个对象也能正确对齐。

计算时固定做四步：

1. 当前偏移从 0 开始。
2. 放每个成员前，把偏移向上调整到该成员对齐要求的倍数。
3. 记录成员起点，再加上成员大小。
4. 所有成员放完后，把总长度向上调整到结构体整体对齐要求的倍数。

向上对齐可以写为 `align(x,K)=ceil(x/K)*K`。K 是二的幂时也常见 `(x+K-1)&~(K-1)`；考试手算直接找下一个倍数即可。

例 D4–E2 真题中的结构体布局

原题截取并展开讲解：2025 期末无答案版（来源资料未随仓库分发），第一大题第4题，PDF 第3页。这里只讨论题中的结构体，不使用该卷其他范围的题。

```
struct S {
    char c;
    int i;
    short s;
    double d;
};
```

字段	大小	放置前偏移	对齐后起点	放置后偏移
c	1	0	0	1
i	4	1	4	8
s	2	8	8	10
d	8	10	16	24

整体对齐为 8，24 已是八的倍数，因此 `sizeof(struct S)=24`。

图4–3 结构体的字节布局。 点表示填充；数字是相对结构体起点的字节偏移。

结构体：填充让每个字段正确对齐

struct S · 总大小 24 字节 · 整体对齐 8



`c@0 → i@4 → s@8 → d@16 → 结束于24`

每个小格代表一个字节；带点的灰格是填充，不是额外字段，也不保证内容为零。

把顺序改为 `double d; int i; short s; char c;` 后，四个成员分别起于 0、8、12、14，实际成员结束于 15，再补一个字节，大小变成 16。声明顺序确实会影响空间；编译器不能擅自把原结构体字段重排成这种顺序。

从汇编反推字段时 先写约束

若 p 在 rdi，`movl 12(%rdi),%eax` 说明读取 p 起点后 12 字节处的四字节数据。它可能对应一个 int 字段，也可能是数组中的 int；只凭这一条指令不一定能确定完整类型。

例如：

```
struct R { char tag; int a[N]; long total; };
```

tag 起于 0，a 起于 4。若 total 起于 24，则 `align(4+4*N,8)=24`，所以 `16 < 4+4*N <= 24`，正整数 N 只能为 4 或 5。**不能凭“24−4”直接除以四得到唯一的 5，因为可能存在填充。** 要得到唯一答案，题目还需给其他约束。

3 联合是共享同一片字节

> Q028：联合这一节在讲什么？为什么写b的字节，也会改变w?

结构体的字段各占一段空间；联合的字段从同一起点开始，相互重叠。联合大小至少容纳最大的字段，并满足整体对齐，不是把各字段大小相加。

例 D4–E3 同一组字节按不同方式解释

助手补充例，按课程的 C 实现和整数表示前提分析：

```
union U {
    unsigned char b[4];
    unsigned int w;
};
```

写入 `b[0..3]={0x11,0x22,0x33,0x44}`。这四次写入改变的是同一个联合的四个字节。

图4–4 共享存储与字节序。 地址从左到右增加；改变解释方式不会重新搬动字节。



小端下最低地址放整数低位字节，因此 w 是 0x44332211；如果题目明确指定大端，w 是 0x11223344。这个联合大小是 4，若改为有这两个字段的结构体，大小则为 8。

位重解释与类型转换不同：把 float 的字节当作整数位模式读取，不等于执行 `(int)f`。后者试图把数值转换成整数。跨类型读写还涉及语言规则；本讲联合题按题给 C 实现处理，不把 `*(float *)&some_int` 当成普遍合法的 C 写法。查看对象字节可使用 `unsigned char *`，复制位模式可使用 `memcpy`。

4 浮点数在汇编里走另一组寄存器

整数与指针通常走通用寄存器，标量浮点参数走 XMM 寄存器。这是两套分别计数的参数通道，不是按所有参数的总顺序依次分配通用寄存器。

在本课程 Linux System V 的普通函数调用中，寄存器可容纳的前几个标量浮点参数依次用 xmm0—xmm7；返回 float 或 double 常用 xmm0。XMM 寄存器是调用者保存。更复杂的结构体参数分类和可变参数函数不在这里展开。

指令中的记号	本讲理解方式
`ss`	标量单精度，float
`sd`	标量双精度，double
`movss`、`movsd`	浮点数据传送；结合操作数判断读内存还是写内存
`addss`、`addsd`	相应精度的浮点加法
`cvtss2sd`	float 数值转换为 double
`cvtsi2sd`	有符号整数转换为 double，整数宽度由具体指令形式决定
`ucomiss`、`ucomisd`	标量浮点比较，设置 ZF、PF、CF

例 D4–E4 混合参数如何分配

助手补充例：

```
double combine(int n, double x, float y, double *out) {
    double r = x + (double)y + (double)n;
    *out = r;
    return r;
}
```

这里 n 使用 edi，x 使用 xmm0，y 使用 xmm1，out 使用 rsi。out 是指针，虽然它指向 double，也要走整数/指针参数通道；它是第二个此类参数，不是第四个通用寄存器参数。

一种只展示主体的指令序列是：

```
cvtss2sd %xmm1, %xmm1     # y: float -> double
addsd    %xmm1, %xmm0     # x + y
cvtsi2sdl %edi, %xmm2     # 32位有符号 n -> double
addsd    %xmm2, %xmm0     # 再加 n
movsd    %xmm0, (%rsi)    # 写入 *out，题设 out 有效
ret                       # xmm0 同时承载返回值
```

不要用整数的 `addq` 来做 IEEE 浮点加法。两种数都存在位模式，但数值编码规则不同。

浮点比较还有 NaN：对 `ucomisd %xmm1,%xmm0`，比较的是 xmm0 与 xmm1。结果如下：

关系	ZF	PF	CF
小于	0	0	1
等于	1	0	0
大于	0	0	0

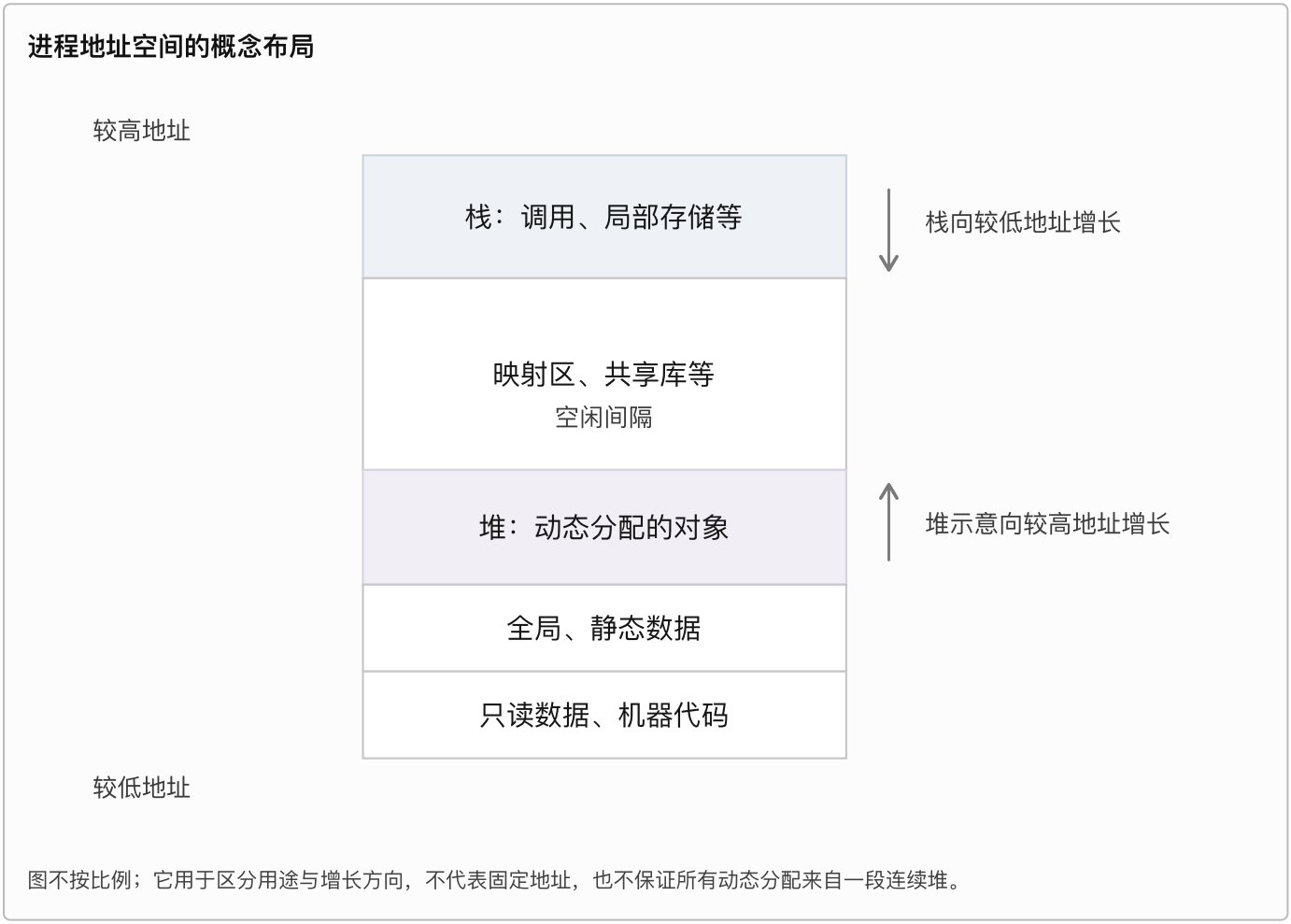
关系	ZF	PF	CF
无序，任一操作数为 NaN	1	1	1

因此只看 ZF=1 不能区分相等和无序。题目若允许 NaN，读代码时还要留意奇偶标志 PF 的分支，例如 `jp`。这里 OF、SF 被清零，不能照搬整数 `jl` 的符号比较套路。

5 程序内存布局与越界

内存布局图描述的是进程的地址空间。具体地址与相对间隔随平台、链接和随机化变化，不要求背某台机器的固定地址。

图4–5 常见布局示意。 这是一张概念图，不按比例；“堆向上、栈向下”是课程常用模型，不代表所有分配都只来自单一连续堆。



一个变量的“作用域”和存储位置是两件事。函数里的局部变量可能在寄存器，也可能在栈；局部指针可以指向堆对象。不要把“函数里出现的所有东西”都画到栈上。

字符串还需要结束的一个零字节

`char buf[8]` 最多容纳七个普通字符和一个 `\0`。`\0` 是一个值为零的字符，不是两个字符反斜杠和零。

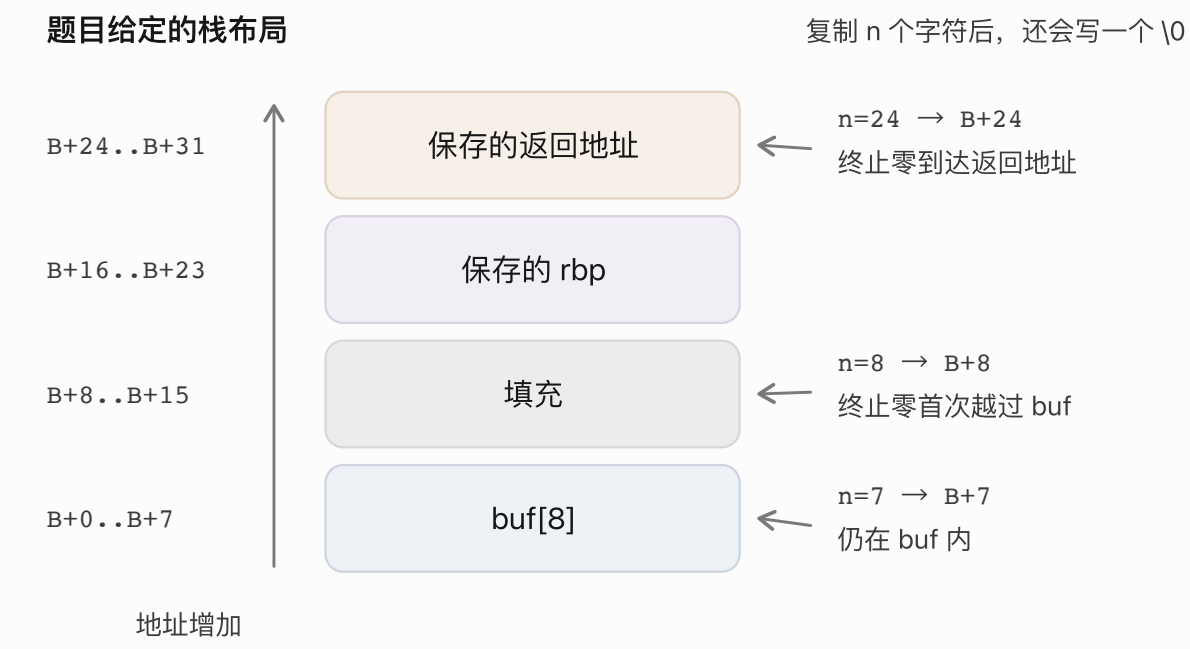
`char a[8]="1234567"` 有终止零；C 中 `char b[8]="12345678"` 的八个位置都被字符占满，没有终止零，不能安全地把 b 当成普通以零结束的字符串传给任意字符串函数。

例 D4–E5 根据给定栈图数越界长度

> Q029：怎样读栈图、数结束零，为什么7、8、24个字符的结果不同?

助手补充例。题目明确给出下列布局；不要把它当成所有编译器的固定布局。

从 buf 写起：终止零也占一个字节



从最低地址 B 向上写；最后一个零字节位于 B+n，所以 n=8 时已经第一次越界。

假设一次不检查边界的字符串复制从 B 开始，依次写入 n 个非零字符，最后再写 `\0`。它总共写 n+1 字节，最后一字节写到 B+n。

- n=7：写 B 到 B+7，恰好容纳，未越界。
- n=8：字符填满缓冲区，终止零写到 B+8，第一次越界已经发生。
- n=24：普通字符写到 B+23，终止零写到返回地址的第一个字节 B+24。

“没有立刻崩溃”不说明写入合法。返回地址被改变后，函数的 ret 可能从栈中取出错误的下一条指令地址。C 层面的越界属于未定义行为；试题给栈布局时，是在分析指定机器模型上的可能效果，不是承诺任何 C 程序都表现一样。

防护方法分别改变了什么

> Q030：长度检查、ASLR、NX和canary分别在防什么?

方法	主要作用	不能据此推出
正确检查长度、使用合适的有界接口	避免越界写入本身	只要函数名有 n 就一定安全
ASLR，地址空间布局随机化	增加预测相关地址的困难	所有越界都会消失
NX/不可执行数据页	阻止直接执行被标为不可执行页中的字节	已有可执行代码不能被重新利用

方法	主要作用	不能据此推出
栈 canary	返回前检查某个保护值是否被破坏，发现特定栈写坏	所有内存错误都会被发现，或已经写坏的数据会自动恢复

`fgets(buf,sizeof buf,stdin)` 在空间参数正确时最多读取 size−1 个字符并终止字符串；还需处理失败、残留输入等使用问题。`strncpy` 并不保证在所有输入长度下都追加 `\0`。canary 通常通过比较“是否相等”发现改变，不是比较数值是否更大。

NX 需要处理器的执行权限检查能力与系统配置配合；现代题设已具备该能力时，启用它不等于必须买一颗新 CPU。遇到旧卷的“是否需要新硬件”，要分清“需要硬件支持”和“当前机器需要更换硬件”。

6 ROP 只需先看懂 ret 链

普通 call 把返回地址压栈，ret 把栈顶八字节取作下一执行地址。ROP 利用同一个机制把已有可执行代码中的短片段连接起来；片段通常以 ret 结束，称为 gadget。它不要求把栈上的数据当指令执行，所以仅有不可执行栈并不足以阻止这种控制流。

本讲只做题目给定片段的状态追踪。代码位置是否可预测取决于具体二进制与随机化配置，不把“所有代码地址固定”当成今天的普遍结论。

例 D4–E6 每次到底弹出什么

助手补充例，假定两个片段如下，STOP 是停止观察的位置：

```
G1: popq %rax
    ret
G2: addq %rax, %rbx
    ret
```

初始 rbx=5，rsp=0x1000，即将执行一个 ret。题给栈内容：

图4-6 返回链与数据槽。 所有槽都是八字节，注意常数 7 是数据，不是跳转地址。

ROP 返回链：初始栈与完整执行状态

0x1018	STOP	G2 末尾 ret 读取
0x1010	G2	G1 末尾 ret 读取
0x1008	7	G1 的 pop 读取，这是数据
0x1000	G1 ← rsp	初始 ret 读取

步骤	rsp	rax	rbx	下一位置
起点：即将执行初始 ret	`0x1000`	未知	5	初始 ret
初始 ret：转到 G1	`0x1008`	未知	5	G1 的 popq %rax
G1 的 pop：rax = 7	`0x1010`	7	5	G1 的 ret
G1 的 ret：转到 G2	`0x1018`	7	5	G2 的 addq %rax,%rbx
G2 的 add：rbx = 12	`0x1018`	7	12	G2 的 ret
G2 的 ret：到达 STOP	`0x1020`	7	12	STOP

1. 本步尚未执行。rsp 指向 G1 所在槽；7 是下一个槽里的数据，不是代码地址。
2. 初始 ret 读出 0x1000 处的 G1，控制转到 G1；rsp 加 8 到 0x1008。
3. pop 从 0x1008 读出数据 7 放入 rax；rsp 再加 8 到 0x1010。控制仍在 G1。
4. ret 从 0x1010 读出 G2，控制转到 G2；rsp 加 8 到 0x1018，rax 仍为 7。
5. add 只把 rbx 改为 5+7=12，不从栈弹出数据，所以 rsp 仍为 0x1018。
6. 最后的 ret 读出 0x1018 处的 STOP；rsp 加 8 到 0x1020，观察在 STOP 结束。

执行步骤	结果	执行后 rsp
初始 ret	下一条指令是 G1	0x1008
G1 的 pop	rax=7	0x1010
G1 的 ret	下一条指令是 G2	0x1018
G2 的 add	rbx=5+7=12	0x1018
G2 的 ret	转到 STOP	0x1020

做题时先按执行顺序列出 pop 与 ret，每次各消耗一个八字节槽。不要把所有栈槽都当作返回地址。

7 独立练习

以下均为助手补充题，数值与示范不同。必做 D4-P1—P8 共40分，32分为过关参考；其中短题每道3分，两道中题各11分。先独立作答，再查看第4讲练习答案。

D4-P1 联合与字节序 3分

`union { unsigned char b[4]; unsigned int w; } u;`。依次写入 b 为 0x01、0x20、0x03、0x40。按本讲的 x86-64 小端前提，求 w；若明确改为大端，求 w；求联合大小。每问1分。

D4-P2 数组与 sizeof 3分

有 `int a[3][4]; int (*p)[4]=a;`。求 `sizeof(a)`、`sizeof(p)`、p+1 比 p 的地址增加的字节数。每问1分。

D4-P3 浮点参数 3分

函数 `double shift(double *p,long k,float x)` 中，p、k、x 在入口时各用哪个寄存器传递？每问1分。额外自检：返回的 double 用哪个寄存器，不另计分。

D4-P4 字符串边界 3分

`char buf[12]` 起于 B，题设保存的返回地址从 B+24 开始。不检查长度地写入 n 个非零字符，再追加零。求：不越界的最大 n；第一个越界的 n；使终止零恰好写到返回地址首字节的 n。每问1分。只做纸面分析，不运行越界代码。

D4-P5 防护识别 3分

分别指出：哪种机制增加地址预测难度；哪种机制检查栈中的保护值是否改变；为何不可执行栈仍不能单独阻止利用已有代码的返回链。每问1分。

D4-P6 返回链追踪 3分

G1 是 `popq %rax; ret`；G2 是 `leaq (%rax,%rax,2),%rax; ret`。初始 rsp=0x800，即将执行 ret；地址 0x800、0x808、0x810、0x818 中依次是 G1、9、G2、DONE。求：进入 G1 时 rsp；执行 G2 的 lea 后 rax；转到 DONE 后 rsp。每问1分。

D4-P7 二维数组中题 11分

`int a[3][5]` 首地址为 0x2000，所有下标合法。

1. 写出 `&a[i][j]` 的一般地址公式。(2分)
2. 求 `&a[2][3]`。(2分)
3. 求 `a+1` 的地址和 `sizeof(a)`。(2分)
4. 假设符号 TAB 指向该数组，i 在 rdi，j 在 rsi，补齐主体片段：(3分，每行1分)

```
leaq (____,____,____), %rax  # 先求5*i
addq ____, %rax
movl TAB(,____,____), %eax
```

5. 若换成 `int *rows[3]`，为什么不能继续无条件使用相同的二维数组地址公式？ (2分)

D4-P8 结构体中题 11分

```
struct Record {
    char c;
    short s;
    long n;
    int a[3];
};
```

1. 求四个字段的偏移。(2分)
2. 求整体对齐和 sizeof。(2分)
3. 若对象首地址为 0x3000，求 a[2] 的地址。(2分)

4. 在 `struct Record r[2]` 中，第二个对象比第一个对象晚多少字节？为何不能只加各字段原始大小之和？（2分）
5. 另有 `struct T { char c; int a[N]; long z; };`，N 为正整数。若 z 的偏移为16，求所有可能的 N，并说明为何不能仅取最大的一个。（3分）

D4–X1 达标后选做

`double update(long *p,double x,float y)` 在指定顺序下计算 `r=(double)*p+x+(double)y` 并返回。写出三个参数和返回值的寄存器；写出读入 *p、转换为 double、两次加法所需的主要指令。假设指针有效，遵守 C 表达式的加法顺序。不计入必做分数。

8 收尾与来源

独立检查五件事：能否从类型确定步长；能否逐字段算对齐；能否从字节序恢复整数；能否区分浮点值和指向浮点的指针；能否按栈槽追踪 ret。错了哪一项，就回到对应的图和例题，再用新数值检验。

本讲的解释、图示和未特别标注的例题属于助手补充；概念依据如下，页码都是 PDF 文件页序，不是教材印刷页：

- 课程第7讲（来源资料未随仓库分发）：第3—23页数组与指针，第25—35页结构体和对齐，第37—42页浮点机器代码，第45—51页声明和 sizeof。
- 课程第8讲（来源资料未随仓库分发）：第3—11页联合，第13—16页内存布局，第18—33页越界，第40—47页防护，第48—53页ROP。
- 教材（来源资料未随仓库分发）：3.8，3.9，3.10.1，3.10.3—3.10.4，3.11相关部分。这里使用小节定位，不混写文件页序和印刷页码。
- 第7讲研讨题（来源资料未随仓库分发）：第1—2页普通研讨与浮点机器编程部分，不采用拓展研讨。